二、Linux 环境高级编程（余宜宸老师）

2.1 谈谈你对函数指针的理解

C 语言中的函数指针是指向函数的指针变量。它允许我们存储函数的地址，并通过这个地址来调用函数。函数指针的定义方式是将"函数声明"中的"函数名"替换为"(*指针变量名)"，且两端的括号不能省略，否则会被解析为函数声明而非函数指针。

回调函数是什么：

回调函数是函数指针的一个重要应用场景。回调函数指的是通过函数指针传递给另一个函数的函数，在特定事件发生时被调用。回调函数允许程序在运行时动态地调整行为，极大地提高了程序的灵活性。

2.2 shell 是什么，试用 shell 脚本有什么好处

Shell 是指一种 C 语言编写的应用程序。主要作用是获取用户输入的指令及参数，调用现有的程序实现用户的意图；用户在没有图形界面的情况下 shell 是用户访问系统内核服务的唯一途径（类似于 Windows 下的 DOS）；

Shell 也是一种程序设计语言。它允许用户将想要执行的命令按照一定的语法格式书写在文本文件 *.sh 中，随后执行 shell 脚本文件中的指令集合，把这种记录指令的脚本文件叫做 shell 脚本程序。Shell 脚本编程跟 JavaScript、PHP、Python 编程形式一样，只要有文本编辑器和执行脚本的解释器就可以了。

简单来说 Shell 是一个用 C 语言编写的程序，可以解释并执行 shell 脚本文件，同时它也是用户使用 Linux 的桥梁；

脚本的好处至少有以下几点：

1）避免开发人员反复进行一些列重复步骤。比如：联网+检查安装包+安装软件+执行命令，如果经常需要做类似的操作，可以考虑将操作写成脚本；

2）提高编译效率。利用编写好 Makefile 文件（shell 脚本），只需要一个 make 命令，整个工程就开始自动编译。因为 Makefile 文件中已经事先指明了源文件的编译顺序、依赖关系等重要的编译信息；

3）不依赖 IDE 就可以实现功能编码，这在陌生的机器（不具备安装软件的权限）上显得尤为重要；

4）方便调用机器下的所有资源。类似于胶水一样，可以将各类资源（命令、程序、脚本等）拼接在一起使用。

华清远见武汉中心嵌入式学科面试宝典

2.3 Makefile 中 # $ @ := ?= += 的含义

$@	表示规则中的目标文件集。在模式规则中，如果有多个目标，那么，"$@"就是匹配于目标中模式定义的集合。
$%	仅当目标是函数库文件中，表示规则中的目标成员名。例如，如果一个目标是"foo.a(bar.o)"，那么，"$%"就是"bar.o"，"$@"就是"foo.a"。如果目标不是函数库文件（Unix下是[.a]，Windows 下是 [.lib]），那么，其值为空。
$<	依赖目标中的第一个目标名字。如果依赖目标是以模式（即"%"）定义的，那么"$<"将是符合模式的一系列的文件集。注意，其是一个一个取出来的。
$?	所有比目标新的依赖目标的集合。以空格分隔。
$^	所有的依赖目标的集合。以空格分隔。如果在依赖目标中有多个重复的，那个这个变量会去除重复的依赖目标，只保留一份。
$+	这个变量很像"$^"，也是所有依赖目标的集合。只是它不去除重复的依赖目标。
$*	这个变量表示目标模式中"%"及其之前的部分。如果目标是"dir/a.foo.b"，并且目标的模式是"a.%.b"，那么，"$*"的值就是"dir /a.foo"。
=	:= 是立即展开的赋值符号。使用这个符号定义的变量，会在定义时立即被展开。所有的变量在定义时就会被计算好。
?=	?= 是条件赋值操作符。如果变量没有被定义过，那么使用?=可以为其赋一个初值。如果该变量已经被定义，那么该赋值操作将被忽略。
+=	+= 是追加赋值操作符。它用于将新的内容追加到已经定义的变量值之后。

2.4 什么是内存泄露、内存溢出，如何防止

	内存泄露	内存溢出

华清远见武汉中心嵌入式学科面试宝典

通俗解释	1）操作系统以为用户还占有某块内存，但用户丢掉了该内存的地址，失去对该内存的使用权； 2）调用 char *p = malloc(100)，但紧接着 p = NULL，那么这 100 个字节将永远得不到访问，除非程序结束，操作系统回收内存。	1）内存不够用，剩余的内存空间不足以满足需求； 2）企图开辟过大的空间，比如： 栈对象 int num[4096][4096]是极易初始化失败的，尤其是在线程函数中出现时。
防止方法	1)在C++中智能指针可以做到堆对象自动释放，这能有效避免内存泄露； 2）使用托管类接管堆对象（锁管理就是一个例子），避免异常中断导致灾难性的资源泄露； 3）能使用栈对象就不要使用堆对象，避免忘记释放堆对象（这个不报错，但有隐患）；	1）采用即用即释放、以时间换空间的原则。（数据库内存占用较高使用频率较低时，可按照打开—访问—关闭的策略访问） 2）养成良好的资源管理习惯和意识，尽可能做到代码逻辑正确的同时，不浪费资源，不闲置资源、不过多占用资源。

2.5 如何检测内存泄漏

内存泄露是指用程序运行过程中户失去对内存的访问权利，最简单的内存泄露如下。直观感受是：随着时间的推移，程序占用的内存在不断的增加。若代码中没有业务逻辑需要积累存储数据，就一定是发生了内存泄露。确定泄露内存的代码位置就需要借助相关的工具/插件/函数。

```
p = malloc(100);
//free(p);
p = NULL; //不经过函数 free(p)就修改 p 的指向，函数 malloc()开辟的 100 个字节就泄露了
```

1）使用 IDE 提供的调试工具跟踪调试，并在软件窗口中监视内存的使用情况；这种方式比较慢，常与断点配合使用，检查部分代码段有效；

2）使用 glibc 库的内置函数 mtrace()/muntrace()来检测可能出现内存的漏的代码段；

3）安装 Valgrind Memcheck 是检测工具（Linux 下一款程序），使用如下命令执行代码即可：valgrind --tool=memcheck --leak-check=full --log-file=./log.txt ./a.out；

4)还有一些检测内存泄露的函数库/工具，如：dmalloc（动态库）、memwatch（动态库）、 mpatrol（内存泄漏检测器）dbgmem（动态库）、 Electric Fence

补充：glibc 库的内置函数 mtrace()用法举例如下（C/C++环境均可）：

```
#include <stdio.h>
#include <stdlib.h> //函数 malloc()依赖的头文件
#include <mcheck.h> //函数 mtrace()依赖的头文件
```

华清远见武汉中心嵌入式学科面试宝典

```c
char *test()
{
    char *p = malloc(100);
    //free(p);
    p = NULL; //直接修改 p 的指向，函数 malloc()开辟的 100 个字节就泄露了
    return p;
}

int main(int argc, const char *argv[])
{
    setenv("MALLOC_TRACE", "test.log", 0); //1.指定环境变量，省略 perror()
    mtrace(); //2.开始内存泄漏检测，检测结果写入文件 test.log
    char *ptr = test(); //3.执行用户的问题代码/函数
    //muntrace(); //4.停止检测
}
```

编译方法：gcc -g test.c

执行方法：mtrace a.out test.log

执行结果：

```
Memory not freed:
-----------------
   Address     Size     Caller
0x08749480     0x64  at /home/ubuntu/test.c:7
```

检查内存泄露的原理：在申请堆内存时将申请记录下来，释放内存时将申请记录抹除，当函数 main()结束时，若申请记录中仍然存在记录就说明先前某个地方申请的堆空间没有释放！从这个意义上来说，在 C++中，重载运算符 new，记录申请内存的信息（内存地址、内存大小、程序路径、函数名、行号）到容器；然后重载运算符 delete，删除容器中的申请记录；并在函数 main()的最后一步检查容器中的信息，也能实现内存泄露的检测。（参考网址：https://www.cnblogs.com/space-place/p/7646691.html）

2.6 Linux 上硬链接和软链接的区别

内存硬链接（Hard Link）

定义：硬链接是指向文件数据块的直接引用。硬链接和原文件共享相同的索引节点（inode）。

特点：

同一文件系统：硬链接只能在同一个文件系统中创建，不能跨文件系统。

华清远见武汉中心嵌入式学科面试宝典

相同权限：硬链接与原文件具有相同的权限和属性，删除任何一个不会影响其他链接。

共享数据：无论是硬链接文件还是原文件，任何一个文件的修改都会反映在另一个文件上，因为它们共享同一数据块。

计数器：删除一个硬链接，inode 的引用计数减 1，只有当引用计数为 0 时，数据才会被删除。

软链接（符号链接，Symbolic Link）

定义：软链接是一个指向文件路径的文件，它包含指向另一个文件的路径信息。

特点：

跨文件系统：软链接可以跨文件系统。

独立属性：软链接与原文件的属性可以不同，并且软链接本身也有独立的 inode。

不共享数据：软链接只保存目标文件的路径，删除目标文件后，软链接仍然存在，但指向的路径无效，成为"断链"。符号标识：在文件列表中，软链接通常带有一个小箭头（->）指向目标文件。

总结：

硬链接：直接指向文件的数据块，共享同一 inode。

软链接：指向文件路径，有独立 inode，可跨文件系统。

2.7 正则表达式

正则表达式（Regular Expression，Regex）是一种用来描述或匹配字符串模式的工具，广泛应用于文本处理、字符串搜索和替换等任务中。

常见的正则表达式语法

字符匹配：

. 匹配任意单个字符。

[] 匹配方括号内的任意一个字符，如 [abc] 匹配 a、b 或 c。

[^] 匹配不在方括号内的任意字符，如 [^abc] 匹配除 a、b、c 以外的任意字符。

数量匹配：

* 匹配前一个字符 0 次或多次。

+ 匹配前一个字符 1 次或多次。

? 匹配前一个字符 0 次或 1 次。

{n} 匹配前一个字符恰好 n 次。

华清远见武汉中心嵌入式学科面试宝典

`{n,}` 匹配前一个字符至少 n 次。

`{n,m}` 匹配前一个字符 n 到 m 次。

位置匹配：

`^` 匹配字符串的开头。

`$` 匹配字符串的结尾。

分组和引用：

`()` 用于分组和提取匹配的子串。

`|` 表示逻辑或（匹配|前后任意一个表达式）。

`\` 用于转义特殊字符或引用分组，如 `\1` 引用第一个分组。

示例：

`^\d{3}-\d{2}-\d{4}$`：匹配格式为"xxx-xx-xxxx"的字符串，适用于美国社会安全号码。

`[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}`：匹配电子邮件地址。

2.8 结构体与联合体的大小计算

计算**结构体**大小的一般步骤：

1.**成员对齐**：每个成员根据其类型有一个对齐要求，这通常是该类型大小的一个倍数。例如，char 类型可能没有对齐要求（即对齐要求为 1），而 int 类型可能要求 4 字节对齐。编译器会根据这些对齐要求来安排成员在结构体中的位置。

2.**填充字节**：为了满足成员的对齐要求，编译器可能会在成员之间或结构体末尾插入填充字节。这些字节不存储任何有用的信息，只是为了确保每个成员都满足其对齐要求。

3.**结构体整体对齐**：除了成员的对齐要求外，结构体本身也有一个整体对齐要求。这通常是结构体中最大成员对齐要求的倍数。如果结构体的总大小不是这个倍数，编译器会在结构体的末尾添加额外的填充字节。

4.**计算总大小**：最后，结构体的总大小是其所有成员大小、成员之间的填充字节以及结构体末尾的填充字节的总和。

计算**联合体**大小的一般步骤：

1.**最大成员原则**：联合体的大小至少与其最大成员的大小相同。这是因为联合体的所有成员都共享同一块内存区域，所以这块内存的大小必须足够大，以便能够存储联合体中最大的成员。

2.**编译器对齐要求**：除了考虑最大成员的大小外，编译器还可能对联合体有额外的对齐要求。这些对齐要求通常基于编译器或目标平台的特性，并且可能会影响联合体的大小。编译器可能会添加额外的填充字节，以确保联合体满足特定的对齐约束。

3.**无额外填充**：与结构体不同，联合体通常不会在成员之间添加额外的填充字节，因为所有成员都共享相同的内存位置。但是，为了满足整体对齐要求，编译器可能会在联合体的末尾添加填充字节。

华清远见武汉中心嵌入式学科面试宝典

2.9 大端字节序与小端字节序并代码判断（指针与联合体两种）

概念：

大端字节序（Big-Endian）：高字节存放在内存的低地址处，低字节存放在内存的高地址处。

小端字节序（Little-Endian）：低字节存放在内存的低地址处，高字节存放在内存的高地址处。

指针方式：

1.定义一个整型变量并赋值：首先，定义一个 int 类型的变量，并给它赋一个值，这个值的二进制表示在内存中会按照系统的字节序来存储。

2.通过指针访问该变量的首字节：然后，定义一个 char 类型的指针，让它指向这个 int 类型的变量。由于 char 类型占用 1 个字节，因此通过 char 类型的指针可以访问 int 变量的首字节。

3.判断首字节的值：最后，判断通过 char 类型指针访问到的首字节的值。**如果这个值等于整个 int 变量值的最低位字节，那么系统就是小端字节序；如果这个值等于整个 int 变量值的最高位字节，那么系统就是大端字节序。**

```
#include <stdio.h>


int main() {
    int num = 1; //  定义一个整型变量并赋值为 1
    char *p = (char *)&num; //  定义一个 char 类型的指针，指向 num 的地址
    if (*p == 1) { //  判断首字节的值
        printf("Little-Endian\n"); //  如果是 1，则是小端字节序
    } else {
        printf("Big-Endian\n"); //  否则是大端字节序
    }
    return 0;
}
```

联合体方式：

1、定义一个联合体，其中包含一个整型成员 a 和一个字符型成员 i。

2、将整型成员 a 赋值为 1。

3、通过访问字符型成员 i 的值来判断字节序。如果 i 的值为 1，表示低地址保存的是整型成员 a 的高位字节，即小端字节序；如果 i 的值为 0，表示低地址保存的是整型成员 a 的低位字节，即大端字节序。

这种方法利用了联合体成员共享内存的特性，通过访问不同成员来判断字节序。这种方法简单且有效，是判断大小端字节序的常用手段之一

华清远见武汉中心嵌入式学科面试宝典

```c
#include <stdio.h>
// 定义一个联合体，包含一个整型成员和一个字符型成员
union EndianTest {
    int a;
    char i;
};
int main() {
    union EndianTest et;
    et.a = 1; // 将整型成员赋值为 1

    // 通过访问字符型成员的值来判断字节序
    if (et.i == 1) {
        printf("Little-Endian\n"); // 如果字符型成员的值为 1，则是小端字节序
    } else {
        printf("Big-Endian\n"); // 否则是大端字节序
    }

    return 0;
}
```

2.10 编写 strcpy 函数（字符串拷贝函数）的实现 函数原型：char *strcpy(char *strDest, const char *strSrc);

函数原型：

```c
char *strcpy(char *strDest, const char *strSrc);
```

参数：

strDest：目标字符串指针，表示要复制到的内存位置。

华清远见武汉中心嵌入式学科面试宝典

strSrc：源字符串指针，表示要复制的字符串。

功能：

将 strSrc 指向的字符串内容复制到 strDest 指向的内存区域，包括字符串末尾的空字符\0，表示字符串的结束。

返回值为 strDest 指针。

注意事项：

strDest 必须有足够的内存空间来存放 strSrc 中的所有字符，否则可能导致内存越界和程序崩溃。

strcpy 不会检查 strDest 的长度，因此使用时要确保 strDest 的空间足够大。

实现：

```
char* strcpy(char* dest, const char* src) {
    char* original_dest = dest; // 保存目标字符串的原始地址

    // 循环复制每个字符，直到遇到源字符串的终止字符 '\0'
    while (*src != '\0') {
        *dest = *src;
        dest++;
        src++;
    }

    // 添加终止字符
    *dest = '\0';

    // 返回目标字符串的原始地址
    return original_dest;
}
```

2.11 编写 strstr 函数（查找字符串子串函数）函数原型：char *strstr(const char *str1, const char *str2);

函数原型：

华清远见武汉中心嵌入式学科面试宝典

```
char *strstr(const char *str1, const char *str2);
```

参数：

str1：要搜索的主字符串。

str2：要查找的子串。

功能：

在字符串 str1 中查找字符串 str2 的第一次出现位置。

如果找到，返回指向 str2 在 str1 中首次出现位置的指针。

如果未找到，返回 NULL。

注意事项：

如果 str2 为空字符串（即""），则 strstr 返回 str1 的起始位置。

str1 和 str2 必须是以空字符\0 结尾的有效 C 字符串。

实现：

```
char* strstr(const char* haystack, const char* needle) {
    // 如果 needle 是空字符串，则返回 haystack 的初始位置
    if (!*needle) {
        return (char*)haystack;
    }
    const char* p1; // 用于遍历 haystack 的指针
    const char* p2; // 用于遍历 needle 的指针
    const char* p1_advance = haystack; // 用于在 haystack 中前进的指针

    // 遍历 haystack 字符串
    for (p1 = haystack; *p1_advance; ++p1) {
        p1_advance = p1; // 重置前进指针到当前遍历位置
        p2 = needle; // 重置 needle 遍历指针到起始位置

        // 当两个字符串的字符都匹配时，继续遍历
        while (*p1_advance && *p2 && *p1_advance == *p2) {
```

华清远见武汉中心嵌入式学科面试宝典

```
            p1_advance++;
            p2++;
        }

        // 如果完整匹配了 needle 字符串，则返回当前 haystack 的位置
        if (!*p2) {
            return (char*)p1;
        }
    }

    // 如果没有找到匹配，返回 NULL
    return NULL;
}
```

华清远见武汉中心嵌入式学科面试宝典
华清远见武汉中心嵌入式学科面试宝典
华清远见武汉中心嵌入式学科面试宝典